PyTorch Conference

Triton compiler

Presenter: Thomas Raoux

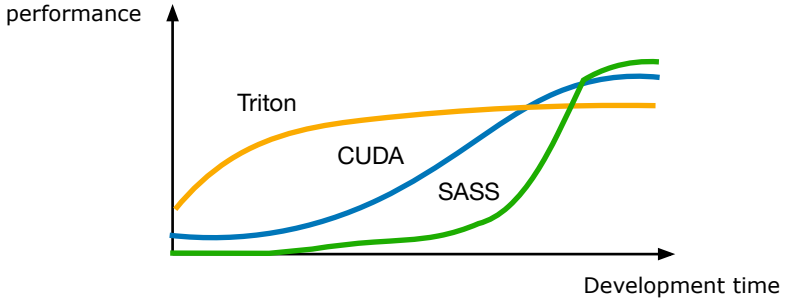
Agenda

- What is triton?
- How does it fit in a DNN stack?
- How does the compiler simplify user’s work?

What is Triton?

Triton is a Python DSL to write machine learning ~~GPU~~ kernels. It is composed of a language and compiler that can be used independently.

Triton allows researchers with no GPU programming experience to write high performance GPU code.

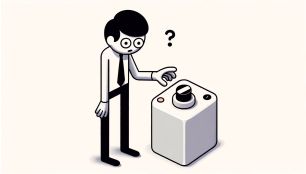


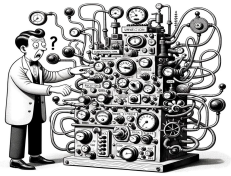
Why do we need a new language?

Pytorch **Triton** **CUDA** **PTX** **SASS**

More compiler control

More user control





Finding the control sweet spot

Triton abstract complex GPU concept but leaves control to user on the algorithm and tuning

	CUDA	Triton	Torch Op
Algorithm	User	User	Compiler
Shared memory	User	Compiler	Compiler
Barriers	User	Compiler	Compiler
Distribution to blocks	User	User	Compiler
Grid size	User	User	Compiler
Distribution to Warps/threads	User	Compiler	Compiler
Tensor Core usage	User	Compiler	Compiler
Coalescing	User	Compiler	Compiler
Intermediate data layout	User	Compiler	Compiler
Workgroup size	User	User	Compiler

Example: softmax

```
@triton.jit
def softmax_kernel(
output_ptr, input_ptr, input_row_stride, output_row_stride, n_cols,
BLOCK_SIZE: tl.constexpr
):
# The rows of the softmax are independent, so we parallelize across those
row_idx = tl.program_id(0)
# The stride represents how much we need to increase the pointer to advance 1 row
row_start_ptr = input_ptr + row_idx * input_row_stride
# The block size is the next power of two greater than n_cols, so we can fit each
# row in a single block
col_offsets = tl.arange(0, BLOCK_SIZE)
input_ptrs = row_start_ptr + col_offsets
# Load the row into SRAM, using a mask since BLOCK_SIZE may be > than n_cols
row = tl.load(input_ptrs, mask=col_offsets < n_cols, other=-float('inf'))
# Subtract maximum for numerical stability
row_minus_max = row - tl.max(row, axis=0)
# Note that exponentiation in Triton is fast but approximate (i.e., think __expf in CUDA)
numerator = tl.exp(row_minus_max)
denominator = tl.sum(numerator, axis=0)
softmax_output = numerator / denominator
# Write back output to DRAM
output_row_start_ptr = output_ptr + row_idx * output_row_stride
output_ptrs = output_row_start_ptr + col_offsets
tl.store(output_ptrs, softmax_output, mask=col_offsets < n_cols)
```

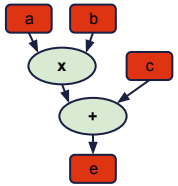
Workgroup/tile id

Load a block of memory. Compiler will make sure this is coalesced

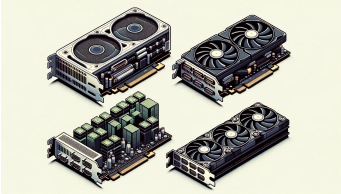
Block level reduction will implicitly use shared memory.

Triton as part of DNN stack

```
a = torch.randn(64, 32)
b = torch.randn(32, 64)
c = torch.randn(64, 64)
d = torch.mm(a, b)
e = c + d
```




```
__global__
void mm(float *a, float *b, float
*c) {
    float *a_tile;
    float *b_tile;
    ...
}
```



Pytorch/TensorFlow/JAX — Model

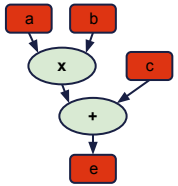
XLA HLO/Torch inductor/TVM — Graph

CUDA/HIP/OneAPI/OpenCL — Kernel

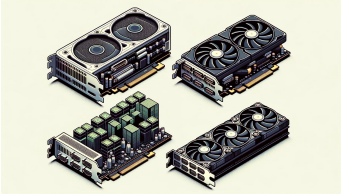
GPU/CPU/ASIC — Hardware

Triton as part of DNN stack

```
a = torch.randn(64, 32)
b = torch.randn(32, 64)
c = torch.randn(64, 64)
d = torch.mm(a, b)
e = c + d
```




```
__global__
void mm(float *a, float *b, float
*c) {
    float *a_tile;
    float *b_tile;
    ...
}
```



Pytorch/TensorFlow/JAX

Triton custom op

Model

XLA HLO/Torch inductor/TVM

Graph

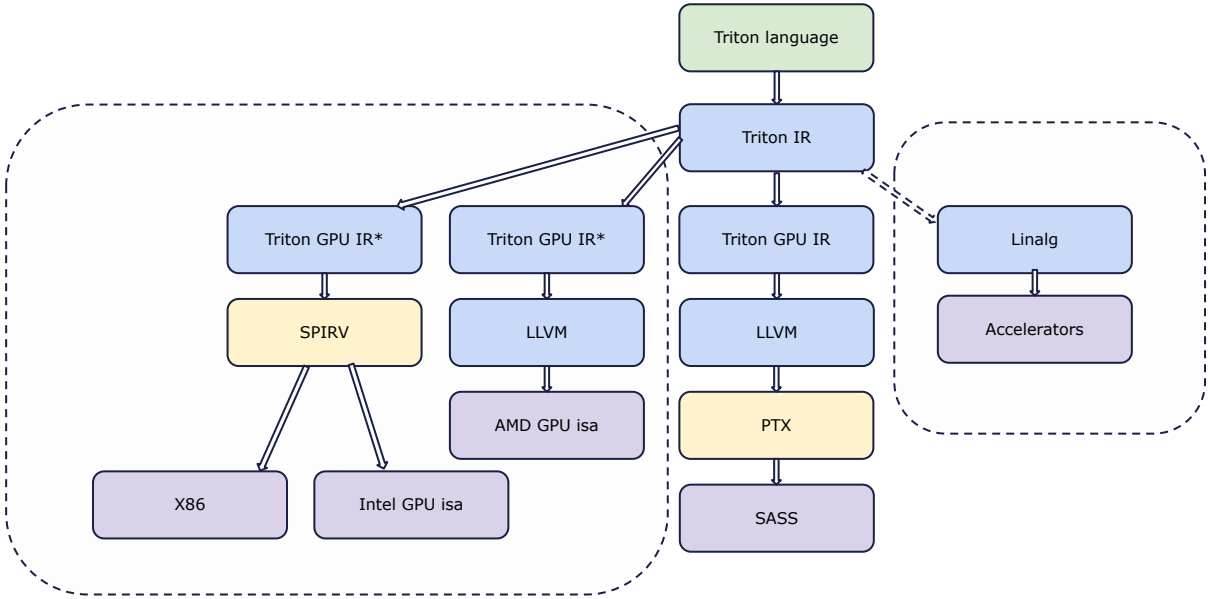
CUDA/HIP/OneAPI/OpenCL/Triton

Kernel

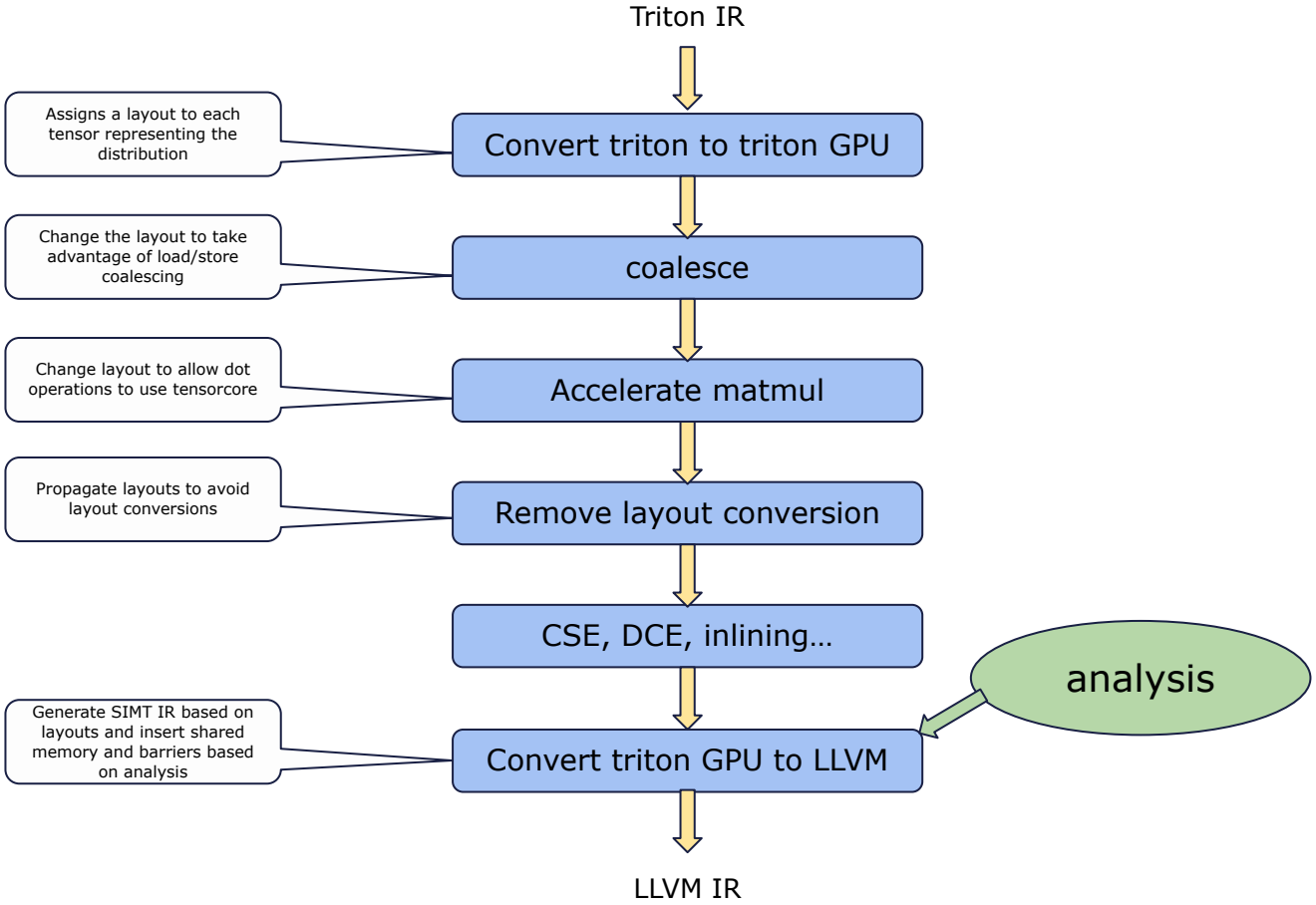
GPU/CPU/ASIC

Hardware

Compiler architecture



Zoom in: compiler pipeline



Contributing

Triton is fully open source: https://github.com/openai/triton

Monthly community meeting info: https://github.com/openai/triton/tree/main/docs/meetups

PyTorch Conference

THE LINUX FOUNDATION